

Prática DevOps: Kubernetes Social Ledger

Disciplina: DevOps

Autor: João Paulo Morais Rangel

1. Visão Geral da Implantação

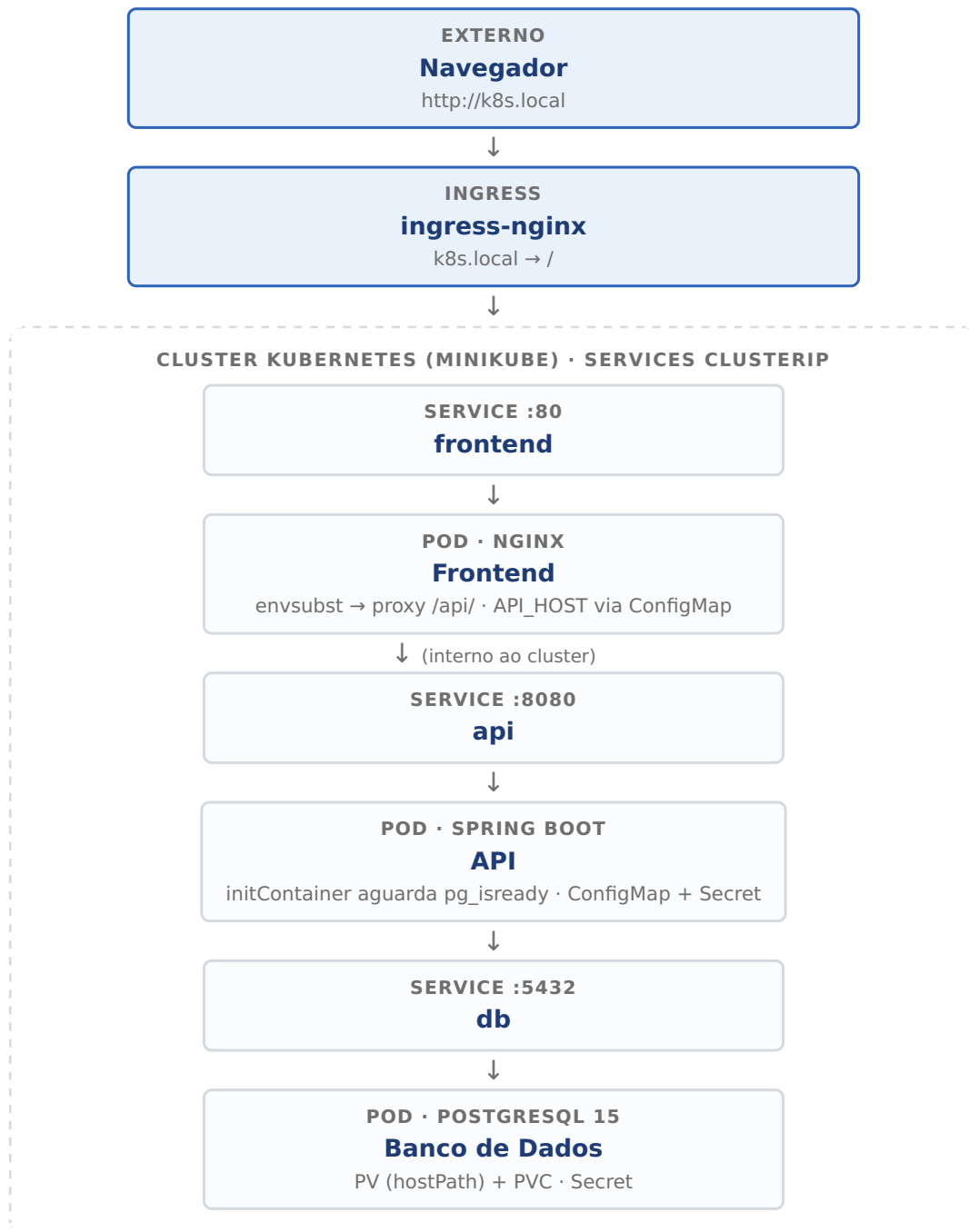
O **Social Ledger** é uma ferramenta full-stack de gestão de despesas compartilhadas, já descrita e containerizada na entrega anterior. Nesta etapa, a mesma aplicação é migrada do Docker Compose para um cluster **Kubernetes local (Minikube)**, utilizando **Helm** como gerenciador de pacotes para automatizar e parametrizar toda a implantação.

A aplicação continua composta pelos três componentes originais — banco de dados PostgreSQL, API Spring Boot e frontend React/Nginx — agora representados como objetos nativos do Kubernetes: Deployments, Services, ConfigMap, Secret, PersistentVolume, PersistentVolumeClaim e Ingress.

Camada	Tecnologia	Descrição / Papel
Backend	Java 21 / Spring Boot 4	API RESTful com arquitetura limpa.
Frontend	React / TypeScript / Vite	SPA servida pelo Nginx como proxy reverso.
Banco de Dados	PostgreSQL 15	Persistência relacional com volume persistente.
Orquestração	Kubernetes / Minikube	Cluster local para implantação dos pods.
Empacotamento	Helm 3	Chart parametrizável para instalar/atualizar toda a aplicação com um único comando.
Roteamento	Nginx Ingress Controller	Exposição da aplicação em <code>http://k8s.local</code> .

2. Arquitetura no Kubernetes

Todos os recursos são implantados no namespace `default` do Minikube. A comunicação entre os pods utiliza exclusivamente DNS interno do cluster, via os Services do tipo **ClusterIP**. O único ponto de entrada externo é o **Ingress**, que roteia o tráfego de `k8s.local` para o Service do frontend. A API e o banco de dados não possuem exposição externa.



2.1. Pod de Banco de Dados (social-ledger-db)

- **Imagem:** `postgres:15-alpine`.
- **Persistência:** `PersistentVolumeClaim` de 1 Gi, vinculado a um `PersistentVolume` explícito, montado em `/var/lib/postgresql/data`. Os dados sobrevivem a reinicializações e recriações do pod.
- **Credenciais:** as variáveis `POSTGRES_DB`, `POSTGRES_USER` e `POSTGRES_PASSWORD` são injetadas a partir do `Secret`, nunca hardcoded no manifest.
- **Health checks:** `readinessProbe` e `livenessProbe` executam `pg_isready`, equivalente ao `healthcheck` do Docker Compose.

2.2. Pod de API (social-ledger-api)

- **Imagem:** construída com Multi-stage build (Maven + JRE Alpine) e publicada no Docker Hub.
- **Dependência:** um `initContainer` bloqueia a inicialização do pod até o PostgreSQL responder ao `pg_isready` — equivalente ao `depends_on: condition: service_healthy` do Compose.
- **Configuração:** `SPRING_DATASOURCE_URL` e `SPRING_JPA_HIBERNATE_DDL_AUTO` vêm do ConfigMap; as credenciais `SPRING_DATASOURCE_USERNAME` e `SPRING_DATASOURCE_PASSWORD` são injetadas do Secret via `secretKeyRef`.
- **Health checks:** `readinessProbe` e `livenessProbe` via `tcpSocket` na porta 8080.

2.3. Pod de Frontend (social-ledger-frontend)

- **Imagem:** construída com Multi-stage build (Node/Vite + Nginx Alpine) e publicada no Docker Hub.
- **Configuração via envsubst:** a imagem oficial do Nginx processa na inicialização um template em `/etc/nginx/templates/`, substituindo a variável de ambiente `API_HOST` (fornecida pelo ConfigMap) pelo nome do Service. Isso configura o proxy reverso de `/api/` para `social-ledger-api:8080` via DNS interno, sem necessidade de ConfigMap de arquivo e eliminando problemas de CORS no navegador.
- **Health checks:** `readinessProbe` e `livenessProbe` via `httpGet /`.

3. Artefatos Kubernetes

Todos os artefatos abaixo são templates Helm localizados em `helm/social-ledger/templates/`. Os nomes dos recursos são prefixados com o nome da release Helm (padrão: `social-ledger`), tornando possível instalar múltiplas instâncias no mesmo cluster.

3.1. ConfigMap — configmap.yaml

Centraliza as variáveis de ambiente **não sensíveis** da aplicação, desacoplando a configuração das definições dos Deployments. Os pods consomem esses valores via `configMapKeyRef`, o que permite alterar a configuração sem editar os manifests dos pods. Dados sensíveis (usuário e senha do banco) ficam no Secret.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: social-ledger-config
data:
  SPRING_DATASOURCE_URL: "jdbc:postgresql://social-ledger-db:5432/socialledger"
  SPRING_JPA_HIBERNATE_DDL_AUTO: "update"
  API_HOST: "social-ledger-api"
```

3.2. Secret — secret.yaml

Armazena as credenciais do banco de dados (`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`) codificadas em Base64 com o tipo `opaque`. Os Deployments do banco e

da API referenciam esses valores via `secretKeyRef`, garantindo que senhas nunca apareçam em texto claro nos manifests dos pods.

```
apiVersion: v1
kind: Secret
metadata:
  name: social-ledger-db-secret
type: Opaque
data:
  postgres-db:      "c29jaWFsbGVkZ2Vy"    # socialledger (base64)
  postgres-user:    "cG9zdGdyZXM="        # postgres (base64)
  postgres-password: "cG9zdGdyZXM="        # postgres (base64)
```

3.3. PersistentVolume — db/pv.yaml

O volume de dados do banco é **declarado explicitamente**, em vez de depender do provisionamento dinâmico do Minikube. Isso dá ao volume um ciclo de vida independente da aplicação: com `persistentVolumeReclaimPolicy: Retain` e um `hostPath` no nó, os dados escritos pelo PostgreSQL permanecem no disco mesmo após um `helm uninstall`, sendo reaproveitados em uma nova instalação. O `storageClassName` próprio (`social-ledger-manual`) evita o provisionamento dinâmico e serve de chave para o vínculo estático com o PVC.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: social-ledger-db-pv
spec:
  capacity:
    storage: 1Gi
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: social-ledger-manual
  hostPath:
    path: /data/social-ledger-db
```

3.4. PersistentVolumeClaim — db/pvc.yaml

Solicita ao cluster **1 Gi** de armazenamento com modo de acesso `ReadWriteOnce`. Ao declarar o mesmo `storageClassName` do PersistentVolume (`social-ledger-manual`), o PVC é vinculado **estaticamente** ao PV acima, em vez de disparar provisionamento dinâmico. O Deployment do banco monta este PVC no caminho de dados do PostgreSQL.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: social-ledger-db-pvc
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: social-ledger-manual
  resources:
    requests:
      storage: 1Gi
```

3.5. Deployments

Um **Deployment** é o objeto responsável por declarar o estado desejado de um conjunto de pods — quantas réplicas rodar, qual imagem usar, variáveis de ambiente, volumes e sondas de saúde. O Kubernetes garante continuamente que o estado real do cluster corresponda ao estado declarado.

Deployment	Imagem	Recurso especial
<code>social-ledger-db</code>	<code>postgres:15-alpine</code>	Monta o PVC; credenciais via Secret.
<code>social-ledger-api</code>	<code><user>/social-ledger-api:latest</code>	<code>initContainer</code> aguarda banco; env via ConfigMap + Secret.
<code>social-ledger-frontend</code>	<code><user>/social-ledger-frontend:latest</code>	<code>API_HOST</code> via ConfigMap; Nginx configurado por <code>envsubst</code> .

Todos os Deployments possuem `readinessProbe` (o pod só recebe tráfego quando pronto) e `livenessProbe` (o pod é reiniciado automaticamente se travar), além de `resources.requests` e `resources.limits` para controle de CPU e memória.

3.6. Services

Um **Service** expõe um conjunto de pods sob um endereço DNS estável dentro do cluster. Todos os três Services são do tipo **ClusterIP** — acessíveis apenas internamente. O Service do frontend é o único referenciado pelo Ingress.

Service	Porta	Consumidor
<code>social-ledger-db</code>	5432	API (via <code>SPRING_DATASOURCE_URL</code>)
<code>social-ledger-api</code>	8080	Frontend (<code>nginx proxy_pass</code>)
<code>social-ledger-frontend</code>	80	Ingress

3.7. Configuração do Nginx via envsubst

Em vez de um ConfigMap montado como volume, a configuração do Nginx é resolvida pela própria imagem. A imagem oficial `nginx:alpine` processa automaticamente, na inicialização, qualquer arquivo com extensão `.template` em `/etc/nginx/templates/`: ela substitui as variáveis de ambiente e grava o resultado em `/etc/nginx/conf.d/`. Isso elimina um artefato Kubernetes e torna o container autocontido.

O template usa `${API_HOST}` no `proxy_pass`, e o valor é injetado por variável de ambiente vinda do ConfigMap — `social-ledger-api` no Kubernetes e `api` no Docker Compose. A mesma imagem serve os dois ambientes.

```
# nginx.conf (template)
location /api/ {
    proxy_pass http://${API_HOST}:8080/api/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# Dockerfile
COPY nginx.conf /etc/nginx/templates/default.conf.template

# Deployment (frontend) — valor vem do ConfigMap
env:
  - name: API_HOST
    valueFrom:
      configMapKeyRef:
        name: social-ledger-config
        key: API_HOST
```

3.8. Ingress — ingress.yaml

O **Ingress** é o único ponto de entrada externo da aplicação. Utilizando o controlador `nginx` habilitado como addon do Minikube, roteia todo tráfego recebido para o Service do frontend na porta 80. A API **não é exposta diretamente** pelo Ingress — permanece acessível apenas internamente via o proxy reverso do Nginx no pod do frontend.

O host é parametrizado como uma **lista** (`ingress.hosts`), permitindo expor a aplicação sob múltiplos hostnames (ex.: `k8s.local` e `devops.local`) sem alterar os templates — basta ajustar o `values.yaml`.

```
# values.yaml
ingress:
  hosts:
    - k8s.local
    - devops.local

# ingress.yaml (trecho)
rules:
  {{- range .Values.ingress.hosts }}
  - host: {{ . | quote }}
    http:
      paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: social-ledger-frontend
              port:
                number: 80
  {{- end }}
```

4. Helm Chart

O **Helm** é o gerenciador de pacotes do Kubernetes. Um *chart* é um conjunto de templates parametrizáveis que, ao serem instalados, geram todos os objetos Kubernetes necessários

para a aplicação. Os valores configuráveis (imagens, credenciais, recursos) são centralizados em `values.yaml`.

Estrutura do Chart

```
helm/social-ledger/
├─ Chart.yaml           # metadados (nome, versão, descrição)
├─ values.yaml          # valores padrão configuráveis
├─ templates/
│   ├── _helpers.tpl    # funções reutilizáveis (nomes, labels)
│   ├── configmap.yaml  # variáveis de ambiente não sensíveis
│   ├── secret.yaml     # credenciais do banco (Base64)
│   ├── ingress.yaml    # Ingress global (multi-host)
│   └─ db/
│       ├── deployment.yaml
│       ├── service.yaml
│       ├── pv.yaml
│       └─ pvc.yaml
│   ├── api/
│       ├── deployment.yaml
│       └─ service.yaml
│   └─ frontend/
│       ├── deployment.yaml
│       └─ service.yaml
```

Estrutura por componente: os templates são organizados em subdiretórios por serviço (`db/` , `api/` , `frontend/`), facilitando a adição de novos microserviços sem poluir os demais.

| 5. Manual de Instalação e Execução

Pré-requisitos

- Docker instalado e em execução.
- Minikube instalado (`minikube start` já executado).
- Helm 3 instalado.
- Conta no Docker Hub com as imagens publicadas.

Passo 1 — Configurar o usuário Docker Hub

Abra o arquivo `helm/social-ledger/values.yaml` e substitua `jpmrangel` pelo seu usuário do Docker Hub nos campos `api.image.repository` e `frontend.image.repository`.

Passo 2 — Habilitar o addon Ingress no Minikube

```
minikube addons enable ingress
```

Aguarde o controlador do Ingress ficar no estado `Running` :

```
kubectl get pods -n ingress-nginx -w
```

Passo 3 — Build, push e carga no Minikube

O script `build.sh` na raiz do projeto automatiza as três etapas: build das imagens Docker, envio ao Docker Hub e carregamento local no Minikube.

```
chmod +x build.sh
./build.sh <seu-usuario-dockerhub> latest
```

Passo 4 — Instalar o Helm Chart

```
helm upgrade --install social-ledger ./helm/social-ledger
```

Aguarde todos os pods ficarem no estado `Running`:

```
kubectl get pods -w
```

Passo 5 — Configurar resolução de nome local

Adicione o IP do Minikube ao arquivo `/etc/hosts` (repita para cada host configurado em `ingress.hosts`):

```
echo "$(minikube ip) k8s.local" | sudo tee -a /etc/hosts
```

Passo 6 — Acessar a aplicação

Abra o navegador em: **`http://k8s.local`**

| 6. Roteiro de Testes (Passo a Passo)

Para validar o funcionamento completo da aplicação no cluster Kubernetes, siga o roteiro abaixo, que cobre todas as camadas do sistema — frontend, API e banco de dados.

1. **Autenticação Inicial:** Acesse `http://k8s.local`. Na tela de login, utilize as credenciais padrão criadas pelo mecanismo de inicialização de dados (*Data Seed*):
 - **E-mail:** `guest@email.com`
 - **Senha:** `guest1234`
2. **Cadastro de Participantes:** Navegue até a seção **Groups** e utilize o formulário de **Register** para cadastrar 3 ou 4 usuários fictícios. Isso valida a escrita no banco de dados via API.
3. **Criação de Grupo:** Crie um novo grupo (ex: "Viagem de Férias") e adicione os participantes recém-cadastrados. Salve o grupo.
4. **Acesso ao Dashboard:** Navegue até a aba **Dashboard** e selecione o grupo criado. O painel deve exibir os saldos zerados e o histórico de transações vazio.
5. **Inclusão de Despesas:** Adicione uma nova transação (ex: "Jantar" · R\$ 300,00). Selecione quem pagou e clique em **Split Equally** para dividir igualmente entre os participantes selecionados. Salve a transação.
6. **Validação do Algoritmo:** Adicione mais uma ou duas despesas alternando o pagador. O Dashboard deve atualizar os saldos líquidos e exibir, na seção de liquidação, o

resultado do algoritmo de *Priority Queues* com o número mínimo de transações para zerar as dívidas do grupo.

7. **Persistência do Volume:** Execute `helm uninstall social-ledger` seguido de `helm upgrade --install social-ledger ./helm/social-ledger`. Graças ao PersistentVolume com `Retain`, os dados cadastrados nos passos anteriores permanecem disponíveis após a reinstalação.

Nota de limpeza de ambiente: Para remover completamente a instalação e apagar os dados persistidos (recomeçando do zero), execute:

```
helm uninstall social-ledger
kubectl delete pv social-ledger-db-pv
minikube ssh -- sudo rm -rf /data/social-ledger-db
```

7. Comandos Úteis

- Verificar status dos pods: `kubectl get pods`
- Ver logs da API em tempo real: `kubectl logs -f deployment/social-ledger-api`
- Ver logs do frontend: `kubectl logs -f deployment/social-ledger-frontend`
- Inspecionar o Ingress: `kubectl describe ingress social-ledger-ingress`
- Verificar volume e claim: `kubectl get pv,pvc`
- Verificar os valores do Helm chart: `helm get values social-ledger`
- Expor múltiplos hosts: `helm upgrade --install social-ledger ./helm/social-ledger --set 'ingress.hosts={k8s.local,devops.local}'`
- Renderizar templates sem instalar: `helm template social-ledger ./helm/social-ledger`